The Journal of Open Source Software

SOFIA: A Python Library for High-Quality 2D Triangular Mesh Adaptation

Youssef Mesri [1]

1 MINES Paris - PSL, France

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 18 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Triangular meshes are fundamental data structures in computational science and engineering, serving as the backbone for numerical simulations in fluid dynamics, structural analysis, heat transfer, and many other fields. The quality of a mesh directly impacts the accuracy, stability, and convergence of numerical solutions (Manzinali et al., 2018; Mesri, Zerguine, et al., 2008; Persson & Strang, 2004). `SOFIA` (Scalable Operators for Field-driven Iso/Ani Adaptation) is a Python library that provides robust and efficient tools for 2D triangular mesh modification, refinement, and quality improvement through local topological operations, with particular emphasis on **anisotropic mesh adaptation** and **automatic boundary preservation**.

Statement of Need

While several mesh generation tools exist (Geuzaine & Remacle, 2009; Kloeckner, 2015; Shewchuk, 1996; Zhou, 2018), there is a gap in the Python ecosystem for a lightweight, well-documented library focused specifically on **mesh adaptation** and **local modification operations**. Existing solutions either require complex dependencies (C/C++ bindings), lack comprehensive documentation, or focus primarily on initial mesh generation rather than adaptive remeshing during simulation workflows.

`SOFIA` addresses these needs by providing:

1. **Pure Python implementation with optional C++ acceleration**: Easy to install and modify, with optional C++ backend for performance-critical operations
2. **Local topological operations**: Fine-grained control over mesh modification through fundamental operations (split, collapse, flip, add, remove)
3. **Anisotropic mesh adaptation**: Metric-based adaptation with automatic handling of highly stretched elements near features (boundary layers, shocks, fronts)
4. **Automatic boundary preservation**: Edge collapse that automatically detects and preserves domain boundaries
5. **Quality-driven adaptation**: Built-in quality metrics and optimization strategies handling both isotropic/anisotropic meshes
6. **Boundary-aware operations**: Safe manipulation of mesh boundaries with conformity preservation
7. **Production-ready reliability**: Extensive test suite (100+ unit tests) ensuring robustness

The library is designed for computational scientists, researchers, and engineers who need to adaptively refine/coarsen meshes during simulations, optimize existing meshes, implement custom mesh adaptation strategies, or generate boundary layer meshes for high-Reynolds number flows.

Functionality

Core Operations

SOFIA implements the fundamental local operations for triangular mesh modification:

- **Edge operations**: Split edges at midpoint or custom locations, collapse edges, and flip edges
- **Vertex operations**: Insert and remove vertices using cavity re-triangulation
- **Pocket filling**: Fill holes in meshes after vertex removal or other operations
- **Boundary operations**: Safely manipulate boundary edges and vertices while maintaining domain conformity
- **Automatic boundary detection**: Edge collapse automatically detect boundary vertices and preserve domain geometry.

Anisotropic Mesh Adaptation

A key feature of SOFIA is its support for anisotropic mesh adaptation, crucial for capturing directional features (Alauzet, 2010; Loseille & Alauzet, 2011; Mesri, Alauzet, et al., 2008; Mesri, Zerguine, et al., 2008; Mesri et al., 2016):

- **Metric tensor field**: User-defined symmetric positive-definite tensor field specifying desired mesh resolution and anisotropy
- **Metric-based edge lengths**: Operations use metric edge length $L_M(e) = \sqrt{(p_2 - p_1)^T M (p_2 - p_1)}$ instead of Euclidean distance
- **Boundary layer support**: Natural support for highly anisotropic elements near boundaries (aspect ratios up to 10:1 or higher)
- **Smooth transitions**: Metric fields can specify smooth transitions from anisotropic to isotropic regions

Key Innovation: Boundary-aware Edge Collapse

Traditional edge collapse operations use the midpoint of the collapsed edge, which can deform domain boundaries. SOFIA implements an edge collapse that:

1. **Automatically detects boundary vertices** using topological information (edges with only one incident triangle)
2. **Preserves boundaries** by collapsing to the boundary vertex instead of the midpoint when one or both endpoints lie on the boundary
3. **Requires no manual vertex protection** the boundary preservation is built into the operation itself
4. **Maintains geometric accuracy** with zero deviation from straight boundaries

This innovation is particularly important for anisotropic remeshing, where many edges near boundaries need to be collapsed while maintaining domain geometry (Mesri et al., 2012).

Boundary Layer Insertion and Adaptation

For high-Reynolds number flow simulations and other applications requiring fine resolution near boundaries, SOFIA provides boundary layer mesh generation capabilities:

- **Structured layer insertion**: Insert vertices at geometric progression distances from boundaries (e.g., $y_i = y_0 \cdot r^i$)
- **Direction-aware metrics**: Compute normal and tangential directions to boundaries for proper metric alignment
- **Progressive refinement**: Smooth transition from very fine resolution at walls to coarse resolution elsewhere

DRAFT

The workflow combines explicit boundary layer construction with metric-driven adaptation for High fedility simulations.

Quality Management

The library provides quality assessment and improvement tools with separate handling for isotropic/anisotropic meshes:

- **Quality metrics**: Minimum angle, area ratios, aspect ratios, and shape measures for triangles
- **Metric-based validation**: For anisotropic meshes, quality assessed using metric edge length distribution rather than geometric angles
- **Quality-driven refinement**: Adaptive refinement based on element size, quality thresholds, or user-defined criteria
- **Optimization strategies**: Vertex smoothing with boundary protection, edge flipping cascades, and iterative quality improvement
- **Validation framework**: Incremental topology checking and optional strict validation mode

Adaptive Refinement Workflows

`SOFIA` supports multiple adaptation strategies for both isotropic/anisotropic meshes:

- **Isotropic refinement**: Uniform or area-based refinement maintaining equilateral triangles
- **Anisotropic refinement**: Metric-driven adaptation with split/collapse/smooth cycles
- **Edge-based refinement**: Selectively refine specific edges (e.g., boundary refinement)
- **Quality-based refinement**: Target low-quality elements for improvement
- **Combined workflows**: Multi-criteria refinement for complex adaptation scenarios
- **Mesh coarsening**: Reduce element count while preserving geometric features and quality
- **Boundary layer generation**: Create highly stretched elements near boundaries for viscous flow simulations

Integration and Extensibility

The modular architecture allows easy integration into existing simulation pipelines:

- **Simple API**: Intuitive mesh editor interface with clear operation semantics
- **Batch operations**: Efficient processing of multiple mesh regions
- **Visualization tools**: Built-in plotting with Matplotlib for analysis and debugging
- **Extensible framework**: Easy to add custom quality metrics or adaptation strategies

Examples

The repository includes comprehensive examples demonstrating various use cases. Each example includes visualization and quantitative metrics. For instance, the simple anisotropic remeshing achieves perfect boundary preservation (max deviation = 0.00e+00) while reducing approximation error by 3×, demonstrating the effectiveness of automatic boundary detection in edge collapse operations (Manzinali et al., 2018; Mesri et al., 2012).

Performance and Testing

`SOFIA` prioritizes reliability and correctness:

- **Comprehensive test suite**: Over 100 unit tests covering all operations and edge cases
- **Continuous validation**: Tests run on every commit to ensure stability
- **Topology verification**: Built-in checks for mesh conformity and validity
- **Benchmarking**: Performance tests for batch operations and large-scale refinement

DRAFT

The library handles meshes ranging from tens to thousands of elements efficiently. For anisotropic remeshing, typical performance on a 1000-element mesh: - 10-20 iterations to convergence - ~300-400 split operations (long edges in metric space) - ~200-300 collapse operations (short edges, with automatic boundary preservation) - Total execution time: <5 seconds on modern hardware

Comparison with Existing Tools

`SOFIA` complements and extends existing tools in the Python ecosystem (Kloeckner, 2015; Logg et al., 2012; Shewchuk, 1996; Zhou, 2018):

- **Triangle/MeshPy**: Focused on initial mesh generation; `SOFIA` specializes in adaptation and modification, with anisotropic support
- **PyMesh**: Requires C++ dependencies; `SOFIA` offers pure Python with optional acceleration
- **CGAL bindings**: Complex API; `SOFIA` provides simpler, Pythonic interface
- **FEniCS/Firedrake mesh tools**: Tightly coupled to FEM frameworks; `SOFIA` is framework-agnostic
- **MMG/BAMG**: C/Fortran anisotropic remeshers; `SOFIA` offers Python integration and automatic boundary preservation

Key differentiators: - **Automatic boundary preservation** in edge collapse (unique feature) - **Adaptive quality checking** (different criteria for isotropic vs anisotropic) - **Pure Python flexibility** with optional C++ performance - **Educational value** with clear, readable implementations

Acknowledgements

We acknowledge contributions from the computational geometry and mesh generation communities, and thank the reviewers for their valuable feedback.

References

Alauzet, F. (2010). Parallel anisotropic 3D mesh adaptation by mesh modification. *Engineering with Computers*, *26*(3), 247–258. https://doi.org/10.1007/s00366-005-0009-3

Geuzaine, C., & Remacle, J.-F. (2009). Gmsh: A 3-d finite element mesh generator with built-in pre-and post-processing facilities. *International Journal for Numerical Methods in Engineering*, *79*(11), 1309–1331. https://doi.org/10.1002/nme.2579

Kloeckner, A. (2015). MeshPy: Simplicial mesh generation from python. In *Website repository*. Mathema. https://mathema.tician.de/software/meshpy/

Logg, A., Mardal, K.-A., & Wells, G. N. (2012). Automated solution of differential equations by the finite element method. *Springer*. https://doi.org/10.1007/978-3-642-23099-8

Loseille, A., & Alauzet, F. (2011). Continuous mesh framework part i: Well-posed continuous interpolation error. *SIAM Journal on Numerical Analysis*, *49*(1), 38–60. https://doi.org/10.1137/090754078

Manzinali, G., Hachem, E., & Mesri, Y. (2018). Adaptive stopping criterion for iterative linear solvers combined with anisotropic mesh adaptation, application to convection-dominated problems. *Computer Methods in Applied Mechanics and Engineering*, *340*, 864–880. https://doi.org/10.1016/j.cma.2018.06.025

Mesri, Y., Alauzet, F., Loseille, A., Hascoët, L., Koobus, B., & Dervieux, A. (2008). Continuous mesh adaptation models for CFD. *Computational Fluid Dynamics Journal*, *16*(4), 346–355.

DRAFT

Mesri, Y., Guillard, H., & Coupez, T. (2012). Automatic coarsening of three dimensional anisotropic unstructured meshes for multigrid applications. *Applied Mathematics and Computation*, *218*, 10500–10519. https://doi.org/10.1016/j.amc.2012.04.014

Mesri, Y., Khalloufi, M., & Hachem, E. (2016). On optimal simplicial 3D meshes for minimizing the hessian-based errors. *Applied Numerical Mathematics*, *109*, 235–249. https://doi.org/10.1016/j.apnum.2016.07.007

Mesri, Y., Zerguine, W., Digonnet, H., Silva, L., & Coupez, T. (2008). Dynamic parallel adaption for three dimensional unstructured meshes: Application to interface tracking. In *Proceedings of the 17th international meshing roundtable* (pp. 195–212). Springer. https://doi.org/10.1007/978-3-540-87921-3_12

Persson, P.-O., & Strang, G. (2004). A simple mesh generator in MATLAB. *SIAM Review*, *46*(2), 329–345. https://doi.org/10.1137/S0036144503429121

Shewchuk, J. R. (1996). Triangle: Engineering a 2D quality mesh generator and delaunay triangulator. *Applied Computational Geometry Towards Geometric Engineering*, 203–222. https://doi.org/10.1007/BFb0014497

Zhou, Q. (2018). PyMesh: Geometry processing library for python. In *GitHub repository*. GitHub. https://github.com/PyMesh/PyMesh

DRAFT